NEOLABS

SECURITY LABS · SOC LEVEL 1

Wazuh Deployment and Investigation Guide

Architecture, dashboard workflow, workstation setup, recovery and troubleshooting

Version: week1-prod-48

Publication date: 2026-08-12

Classification: Student Training Material

Contents

Source: docs/wazuh-handbook/01-wazuh-architecture-and-neolabs-deployment.md

Wazuh SOC Level 1 Handbook — Module 1

Architecture, Data Flow and the NeoLabs Student Deployment

NeoLabs tested baseline: Wazuh 4.14.7
Module version: 0.2-draft
Research and review date: 1 August 2026
Audience: Beginner-to-intermediate SOC Level 1 interns

> Wazuh is not one program running in one window. It is a set of components that collect, analyse, store and display security data. Understanding which component performed each action is essential for troubleshooting and investigation.

Learning objectives

By the end of this module, a learner should be able to:

- explain the roles of the Wazuh agent, server/manager, indexer and dashboard;
- describe how a source event becomes a decoded event, alert and searchable document;
- distinguish event collection, decoding, rule evaluation, indexing and visualisation;
- explain the difference between Wazuh alerts and archived events;
- identify the ports commonly used by the standard Wazuh components and explain the NeoLabs exposure restrictions;
- describe the local containerised student deployment;
- explain how VCC pod telemetry reaches the learner without installing an agent in the pod;
- identify which settings the learner controls and which pod-scope settings remain operator-controlled;
- perform a basic health and data-flow check without exposing secrets.

1. What Wazuh is

Wazuh is an open-source security platform that provides capabilities such as:

- log collection and analysis;
- security-event detection;
- file integrity monitoring;
- security configuration assessment;
- vulnerability-detection context;
- malware and rootkit-related monitoring;
- cloud and container monitoring;
- inventory and compliance-oriented views;
- agent management and dashboard investigation.

In practical SOC work, Wazuh acts as a security telemetry and detection platform. It can collect events from agents, files, Windows channels, syslog and supported integrations; decode and evaluate those events; forward alerts for indexing; and present them to analysts in the dashboard.

Plain-language analogy

Think of a Wazuh deployment as a security newsroom:

- **Agent or collector:** gathers reports from the field.
- **Manager:** reads the reports, extracts important facts and applies editorial rules about what deserves attention.
- **Indexer:** organises the reports so they can be found quickly.
- **Dashboard:** gives analysts an interface for searching, reviewing and visualising the organised information.

The analogy is useful, but remember that a security event can be delayed, duplicated, incorrectly parsed or absent. The analyst must validate the pipeline.

---

2. Core Wazuh components

2.1 Wazuh agent

A **Wazuh agent** is software installed on a monitored endpoint. Depending on configuration, it can collect or produce information such as:

- operating-system and application logs;
- Windows Event channels;
- file-integrity changes;
- system inventory;
- security configuration assessment results;

- selected command or audit output;
- malware or rootkit-related observations;
- agent health and status information.

The agent normally communicates with the Wazuh server over an authenticated channel.

Important analyst limitation

An agent can report only what its operating system, applications and Wazuh configuration make available. An active agent does not guarantee that every required event category is enabled or correctly parsed.

VCC boundary

SOC interns do **not** install, enrol or administer Wazuh agents inside VCC pods. The pods remain operator-managed. The approved VCC log exporter creates a separate synthetic telemetry feed that is delivered to the learner's local collector.

2.2 Wazuh server or manager

The **Wazuh server** includes the manager processes responsible for receiving, decoding, analysing and coordinating Wazuh data.

Important functions include:

- receiving agent or approved collector data;
- pre-decoding common message information;
- applying decoders to extract fields;
- applying rules and correlation conditions;
- writing alert and archive records;
- managing agent registration and grouping where used;
- exposing the Wazuh server API;
- coordinating supported integrations and active-response functions.

The container in the NeoLabs stack is named:

```
wazuh.manager
```

Manager is not the index

The manager analyses events, but analysts normally search indexed alert documents through the dashboard. When troubleshooting, distinguish "the manager generated the alert" from "the indexer stored and returned the alert."

2.3 Filebeat in the Wazuh server container

The official single-node Wazuh container topology includes Filebeat configuration in the manager container. Filebeat forwards Wazuh alert data securely to the Wazuh indexer.

A problem between manager and indexer can produce this situation:

```
Manager alert file contains the alert
        but
Dashboard search does not show it
```

That difference is a useful troubleshooting clue.

2.4 Wazuh indexer

The **Wazuh indexer** is the search and storage component based on OpenSearch technology. It stores documents in indices and makes them searchable.

The indexer handles:

- index creation and storage;
- field mappings;
- distributed search functions;
- security and access-control configuration;
- queries and aggregations;
- data retention through index-management policies where configured.

The NeoLabs Compose service is:

```
wazuh.indexer
```

Field mapping matters

An indexed value can be mapped as a date, number, IP address, keyword or analysed text. Mapping affects equality, range, sorting and aggregation behaviour. A field being visible in JSON does not guarantee that every query type will work correctly.

2.5 Wazuh dashboard

The **Wazuh dashboard** is the browser interface for:

- security-event and alert investigation;
- agent and server management views;
- security modules and compliance-oriented dashboards;
- Wazuh Query Language filters in supported areas;
- OpenSearch-backed Discover and visualisation functions;

- saved searches and dashboards;
- component configuration available to the logged-in role.

The NeoLabs service is:

```
wazuh.dashboard
```

The standard student profile publishes only the dashboard to the host and binds it to loopback:

```
127.0.0.1:8443
```

This means another device cannot normally reach the dashboard over the network unless the learner or operator deliberately changes the profile. Such a change is not authorised by default.

---

3. How Wazuh analyses an event

A simplified Wazuh analysis flow is:

```
Raw event
  → pre-decoding
  → decoder matching and field extraction
  → rule evaluation
  → alert or no alert
  → alert file
  → Filebeat forwarding
  → indexer document
  → dashboard search and visualisation
```

3.1 Raw event

A raw event is the source record received by Wazuh.

Synthetic VCC example:

```
{
  "schema_version": "1.0",
  "event_id": "evt-auth-0009",
  "event_time": "2026-08-01T09:18:07Z",
  "pod_id": "pod-03",
  "event_type": "authentication",
  "action": "login",
  "outcome": "success",
  "user": "svc-backup",
  "source_ip": "203.0.113.44",
  "session_id": "sess-suspect-902",
  "synthetic": true
}
```


3.2 Pre-decoding

Pre-decoding identifies common header information in supported text formats, such as timestamps, hostnames and program names. Structured JSON may already carry these values as fields.

3.3 Decoding

A **decoder** identifies the event format and extracts named fields.

The VCC collector writes NDJSON. The Wazuh manager reads each JSON line using `log_format=json`, and Wazuh's JSON decoder can expose dynamic fields such as:

```
pod_id
event_type
outcome
user
source_ip
session_id
```

Decoder success does not equal detection

A record can be decoded correctly and still generate no alert because no rule condition was satisfied or because the matching rule's level is below the alert threshold.

3.4 Rule evaluation

A **rule** evaluates decoded information. A rule can match:

- one field or message pattern;
- several fields together;
- an event that follows an earlier rule match;
- a repeated event count within a time window;
- the same user, source, destination or another correlation value;

- child rules that inherit context from a parent rule.

The NeoLabs rules use custom IDs in the documented Wazuh custom range and currently cover synthetic authentication, access-control and telemetry-health events.

Rule result is a detection decision

When a rule matches, it tells the analyst that the configured condition was observed. It does not automatically prove malicious intent or successful impact.

3.5 Alert creation

Rules at or above the configured alert threshold create alert records. The alert includes Wazuh metadata such as:

- rule ID;
- rule level;
- rule description;
- rule groups;
- decoded fields;
- source location;
- manager or agent context;
- timestamp.

3.6 Indexing

Filebeat forwards alerts to the indexer. The indexer creates searchable documents, usually under an index pattern such as:

```
wazuh-alerts-*
```

3.7 Dashboard display

The dashboard queries indexed documents. Filters, selected time field, browser time zone and data view affect what is displayed.

4. Alerts and archives

4.1 Alert data

Alert data contains events that matched Wazuh rules at or above the configured alerting level.

Useful for:

- alert triage;
- rule and severity review;
- dashboards and common investigations;
- tracking detection volume.

Limitation: events that did not become alerts are absent.

4.2 Archive data

Wazuh archives can contain all received events when archive logging and indexing are enabled.

Useful for:

- finding context that did not trigger a rule;
- validating rule coverage;
- searching benign or low-level activity;
- investigating parser and detection gaps.

Costs and risks:

- much higher storage volume;
- more sensitive raw content;
- additional privacy and retention requirements;
- increased indexer workload.

NeoLabs will enable indexed archives only after resource, privacy and retention testing. The handbook must not instruct interns to assume archive data is always present.

---

5. Common ports and the NeoLabs restrictions

Official Wazuh deployments commonly use ports such as:

Port	Common purpose	NeoLabs student profile
`1514/TCP`	Wazuh agent event communication	Not published to the host by default

Port	Common purpose	NeoLabs student profile
`1515/TCP`	Agent enrolment service	Not published to the host by default
`514/UDP` or TCP	Syslog, where configured	Not published in the initial student profile
`55000/TCP`	Wazuh server API	Internal only; not host-published
`9200/TCP`	Wazuh indexer/OpenSearch API	Internal only; not host-published
`5601/TCP` inside container	Dashboard service	Mapped to loopback `8443` on the host

Why the restrictions exist

Publishing a service means making it reachable from the host network. Unnecessary exposure increases the attack surface and can reveal administrative APIs or indexed data.

The initial student stack uses a private Compose network for Wazuh components. Only the VCC collector has outbound egress to the approved telemetry endpoint.

---

6. NeoLabs container topology

```
Host workstation
│
├─ 127.0.0.1:8443
│    └─ wazuh.dashboard
│          ├─ internal TLS → wazuh.indexer
│          └─ internal TLS → wazuh.manager API
│
├─ wazuh.manager
│    ├─ reads /var/ossec/logs/vcc/vcc-events.ndjson
│    ├─ JSON decoding
│    ├─ NeoLabs custom rules
│    └─ Filebeat → wazuh.indexer
│
├─ wazuh.indexer
│    └─ internal-only indexed alert storage
│
└─ vcc.telemetry.collector
     ├─ outbound HTTPS/mTLS only
     ├─ server-issued pod scope
     ├─ synthetic-event validation
     └─ shared volume → manager input file
```

Persistence

Docker named volumes retain Wazuh configuration, index data, queues and dashboard state across ordinary stops. The guarded reset script removes volumes only after explicit destructive confirmation.

Generated files

Official Wazuh configuration and certificates are generated into ignored local directories. They are not committed because they include environment-specific cryptographic material and generated configuration.

---

7. VCC pod telemetry without pod access

The VCC model separates **telemetry access** from **infrastructure access**.

The intern receives:

- a local Wazuh environment;
- a short-lived enrolment token;
- a client certificate issued for the operator-recorded pod;
- synthetic events exported from that pod's scenario.

The intern does not receive:

- pod SSH access;
- an AWS role;
- database credentials;
- container access;
- private pod routes;
- a pod log-file mount;
- the ability to select another pod.

End-to-end scope enforcement

```
Operator assignment
  → one-time token
  → locally generated private key
  → certificate bound to assignment
  → Nginx mTLS verification
  → API lookup of active assignment
  → database query restricted to assigned pod
  → collector verifies response pod and every event pod
```

This is stronger than a shared password because each credential is unique, revocable and tied to an installation and assignment.

---

8. Local setup sequence

Run from `wazuh-stack/` after reading its README.

Step 1 — Generate local secrets

```
bash scripts/generate-local-secrets.sh
```

Creates `.env`, the local installation identifier and protected directories.

Step 2 — Receive operator material

The operator provides:

- the VCC HTTPS base URL;
- the public enrolment CA certificate;
- a short-lived token file.

The operator does not provide a pod password.

Step 3 — Prepare official Wazuh files

```
bash scripts/prepare-stack.sh
```

This checks out the exact official Wazuh Docker tag and commit, copies the single-node configuration, generates local indexer password hashes and creates Wazuh TLS certificates.

Step 4 — Run preflight

```
bash scripts/preflight.sh
```

This checks required tools, exact version pins, file permissions, placeholder secrets, dashboard binding, memory guidance and indexer kernel settings.

Step 5 — Enrol

```
bash scripts/enrol-vcc.sh --token-file /protected/path/bootstrap-token.txt
```

The private key remains on the workstation. The client sends no pod selector.

Step 6 — Start

```
bash scripts/start.sh
```

The script validates the Compose model, builds the collector, starts services and waits for health checks.

9. Health checks

Run:

```
bash scripts/health-check.sh
```

Expected output lists:

```
wazuh.manager
wazuh.indexer
wazuh.dashboard
vcc.telemetry.collector
```

Healthy but unenrolled

The collector can report a healthy waiting state before credentials are installed. This means the collector process is functioning; it does not mean VCC telemetry is arriving.

Manager healthy but no alerts

Possible reasons include:

- collector unenrolled or disconnected;
- no new telemetry;
- manager input file absent or unchanged;
- JSON decoding failure;
- rules not loaded;
- events matched only level-zero parent rules;
- Filebeat/indexer problem;
- dashboard time/filter problem.

Health is component-specific. One green component does not guarantee the complete pipeline works.

10. Basic data-flow validation

Use this order:

1. **Collector health:** Is it enrolled and polling successfully?
2. **Collector cursor:** Is the stored cursor advancing?
3. **Shared NDJSON file:** Are new valid lines present in the telemetry volume?
4. **Manager configuration:** Is the VCC localfile input loaded?
5. **Decoder test:** Does `wazuh-logtest` expose expected fields?

6. **Rule test:** Does the synthetic record match the intended rule?
7. **Manager alert file:** Was an alert written?
8. **Filebeat:** Is forwarding healthy?
9. **Indexer:** Is the cluster healthy and index writable?
10. **Dashboard:** Is the correct data view and absolute time range selected?

Do not skip directly to changing rules when the collector is not delivering data.

---

11. Common beginner misunderstandings

“The dashboard created the alert”

The manager’s rule engine created the alert. The dashboard displayed an indexed representation.

“The agent is active, so every log is collected”

Agent connectivity and source coverage are different questions.

“No alert means no event”

The event may have failed to generate, collect, decode, match, forward, index or appear under the selected filters.

“HTTP 200 proves the action was authorised”

A 200 response records a server outcome, not the legitimacy of the requester.

“Changing `POD_LABEL` changes my pod”

`POD_LABEL` is local display information. Authorised scope comes from the server-side assignment and certificate.

“A high rule level proves an incident”

Rule level expresses detection importance configured by the rule author. The analyst must still validate context and impact.

---

12. Guided exercise

Using the synthetic authentication dataset:

1. identify the raw JSON fields;

2. identify which fields the JSON decoder should expose;
3. identify the NeoLabs parent rule and relevant child rules;
4. predict which records should create alerts;
5. explain why a level-zero parent rule may not appear as an alert;
6. run the approved local rule test after the stack is prepared;
7. find the indexed alert in the dashboard;
8. record manager, indexer and dashboard evidence separately;
9. state one failure that could occur at each pipeline stage.

13. Review questions

1. What is the practical difference between the manager and indexer?
2. What does a decoder do?
3. What does a rule match prove?
4. Why can manager alerts exist without appearing in the dashboard?
5. What is the difference between alert and archive data?
6. Why are ports 55000 and 9200 not published in the student profile?
7. How does VCC provide pod telemetry without giving pod access?
8. Why is a unique client certificate stronger than one shared pod password?
9. What does a healthy but unenrolled collector mean?
10. List the ten data-flow validation stages in order.

Authoritative references

- Wazuh official documentation: architecture, Wazuh components, log data analysis and event logging.
- Wazuh official Docker repository, release `v4.14.7`.
- Wazuh official documentation: JSON decoder, custom rules, rule testing, Wazuh indices and API security.
- OpenSearch documentation: indices, mappings and Query DSL.
- Docker documentation: Compose networks, volumes, health checks and container security.
- `research/AUTHORITATIVE_SOURCE_REGISTER.md`.

Source: docs/dashboard-tutorials/01-orientation-and-alert-investigation.md

Wazuh Dashboard Tutorial 01 — Orientation and First Alert Investigation

NeoLabs tested baseline: Wazuh 4.14.7
Tutorial version: 0.2-draft
Review date: 1 August 2026
Audience: SOC Level 1 interns

> Dashboard labels can change between releases. Verify the installed version under the dashboard **About** page before following a screenshot or menu path.

Learning objectives

By the end of this tutorial, a learner should be able to:

- identify the major Wazuh dashboard areas;
- distinguish platform health, agent status, indexed alerts and raw/archive events;
- set and verify the investigation time range;
- open an alert and inspect the event fields that support it;
- add filters, remove inherited filters and pivot to related activity;
- save an investigation view without embedding private information;
- recognise when a missing result may be a data or time-filter problem;
- record evidence and queries in the NeoLabs templates.

---

1. Before opening the dashboard

From `wazuh-stack/`, check local service health:

```
bash scripts/health-check.sh
```

Expected conditions:

- `wazuh.manager` is running and healthy;
- `wazuh.indexer` is running and healthy;
- `wazuh.dashboard` is running and healthy;

- `vcc.telemetry.collector` is either `healthy` or clearly marked `UNENROLLED` while waiting for authorised credentials.

Open the local dashboard address shown by the health script. The standard NeoLabs profile binds the dashboard to:

```
https://127.0.0.1:8443
```

The local browser may show a certificate warning while the workstation deployment uses generated training certificates. Confirm the address is the local loopback address and follow the programme's approved browser procedure. Never bypass a certificate warning for an unknown or public host.

Record the version

After signing in:

1. Open the upper-left navigation menu.
2. Open **About** in the Wazuh section.
3. Record the dashboard package version and revision in your query journal.
4. Confirm it matches the cohort's supported baseline.

Why this matters: a tutorial written for another release may use different names, fields or menu locations.

---

2. Understand the dashboard areas

The Wazuh home area groups information into broad security functions.

Endpoint security

Common views include:

- Configuration Assessment;
- Malware Detection;
- File Integrity Monitoring.

These views depend on the modules and telemetry configured for monitored endpoints. An empty panel may mean no findings, no compatible source, a disabled module or a time/filter issue.

Threat intelligence

Common views include:

- Threat Hunting;

- MITRE ATT&CK-related views;
- vulnerability and intelligence context where configured.

Threat Hunting is useful for reviewing security events and alerts beyond a single prebuilt dashboard.

Security operations

This area provides operational views such as:

- events and alerts;
- policy or compliance views;
- configuration and platform data.

Cloud security

Cloud dashboards appear when the relevant AWS, Azure, Microsoft or other integrations are configured and sending supported data.

Agents management

Agents management → Summary shows enrolled Wazuh agents and status categories such as:

- Active;
- Disconnected;
- Pending;
- Never connected.

Important VCC distinction: the NeoLabs pod feed is delivered through a pod-scoped telemetry collector and read by the local Wazuh manager. It is not permission to install or manage an agent inside a VCC pod. Do not interpret the absence of a pod-named Wazuh agent as proof that no pod telemetry exists.

Server management and app settings

These areas can show manager settings, API connections, configuration and component health. Some actions require administrator permissions and can affect the local deployment. Interns should follow the tutorial and avoid changing settings outside an assigned lab.

---

3. Alert data versus archive data

Wazuh commonly uses different index patterns for different purposes.

Index pattern	Typical purpose	SOC L1 caution
`wazuh-alerts-*`	Alerts generated when Wazuh rules reach the configured alert threshold	Contains detections, not every received event
`wazuh-archives-*`	All events received by the Wazuh server when archive indexing is enabled	Can be high-volume and may not be enabled in every deployment
`wazuh-monitoring-*`	Agent status information	Status history is not the same as security-event data
`wazuh-statistics-*`	Wazuh server performance and processing statistics	Useful for identifying dropped or delayed events

The NeoLabs starter investigation uses `wazuh-alerts-*`. Archive indexing will be enabled only after storage and privacy settings are approved and tested.

Why alerts are not complete evidence

A record can be absent from `wazuh-alerts-*` because:

- it did not match a rule;
- it matched a level below the alert threshold;
- the decoder failed;
- the record arrived outside the selected time range;
- the source never generated or delivered it.

When archive data is available, it helps investigate events that did not become alerts. When it is unavailable, state that limitation.

---

4. Start with the global time filter

A wrong time filter is one of the most common reasons analysts miss evidence.

1. Locate the time picker near the top-right of the relevant dashboard or Discover view.
2. Record the displayed time zone.
3. Choose an absolute range that includes the event's **event time**.
4. Expand the range slightly before and after the alert.
5. Apply the range and confirm the result count changes as expected.

Relative versus absolute time

- **Relative range:** "Last 15 minutes." Useful for live monitoring but changes as time passes.
- **Absolute range:** fixed start and end timestamps. Better for a report that another analyst must reproduce.

For submitted investigations, record the absolute UTC range even when you first used a relative range.

Event time versus ingest time

The indexed `timestamp` used by Wazuh may not be identical to a source's own `event_time` or the collector's `ingest_time`. Inspect all available timestamp fields before building a precise timeline.

---

5. Open an alert correctly

Use the assigned alert view or **Threat Hunting** view.

1. Set the time range.
2. Clear unrelated saved filters.
3. Find the target alert.
4. Expand the alert row or open its detail panel.
5. Inspect the full field list.
6. Record the rule ID, level, description, event time and affected entities.
7. Locate the original or full-log field where available.

Fields to identify in a NeoLabs VCC event

The exact path can vary after indexing, but look for the following source fields:

- `schema_version`;
- `event_id`;
- `event_time`;
- `ingest_time`;
- `pod_id`;
- `event_type`;
- `action`;
- `outcome`;
- `user` or other identity field;
- `source_ip`;
- `session_id`;
- `correlation_id`;
- `synthetic`.

Also record Wazuh-added fields:

- rule ID and level;
- rule groups;
- decoder name;
- manager or agent fields;
- indexed timestamp;

- source location.

Evidence question

For each displayed field ask:

- Did the source generate this value?
- Did a decoder derive it?
- Did enrichment add it?
- Could it be user-controlled?
- Does it directly support the alert description?

---

6. Add filters without losing context

Most dashboard event views let you filter by selecting a field value or adding a filter manually.

Useful first pivots include:

- `pod_id` for the server-issued assigned pod;
- `user` for the affected identity;
- `source_ip` for related activity from the same source;
- `session_id` for actions within one application session;
- `correlation_id` for records linked across components;
- rule ID for repeated detections;
- `event_type` and `outcome` for activity categories.

Include and exclude filters

An include filter narrows to matching records. An exclude filter removes matching records.

Before trusting the result count:

1. inspect every filter pill;
2. check whether a filter is disabled or inverted;
3. check whether the selected data view is correct;
4. record the filter and time range in the query journal.

Example investigation sequence

For a failure-to-success alert:

1. Filter to the assigned `pod_id`.
2. Filter to the affected `user`.
3. Expand the time range to 30 minutes.
4. Sort by event time ascending.

5. Identify failures and any success.
6. Remove the user filter and search the same `source_ip` for other targeted accounts.
7. Pivot from the success to its `session_id`.
8. Look for account, application and authorization events within that session.
9. Search telemetry-health events for the same time window.

Do not reproduce a denied cross-pod request. Document the denial already present in the synthetic evidence.

---

7. WQL is not the same as every dashboard search bar

Wazuh Query Language (WQL) is used in specialised Wazuh tabs and Wazuh server API filtering. Its general structure is:

```
field operator value
```

Common operators include:

```
=   equality
!=  inequality
>   greater than
<   less than
~   like/contains-style matching
```

WQL uses:

```
;   AND
,   OR
()  grouping
```

Example:

```
status=active;os.name=Ubuntu
```

Values containing spaces must be quoted. WQL is case-sensitive.

However, **Explore → Discover** and some alert-search views use index/search syntax provided by the Wazuh dashboard and OpenSearch layer rather than WQL. Do not paste a WQL expression into every search bar and assume the same meaning. The query reference identifies which language belongs to which interface.

---

8. Save a reproducible view

A saved search or dashboard view can preserve:

- selected fields;
- sort order;
- filters;
- query text;
- time range, depending on the interface.

Use a neutral name such as:

```
LAB01-authentication-timeline-student03
```

Do not put passwords, tokens, private URLs or personal information in saved-view names.

In the report, still write the query and time range. A saved object can be changed or deleted and may not be available to the reviewer.

---

9. Screenshot evidence

Take a screenshot only when it adds visual context. A strong screenshot should show:

- the relevant dashboard/view name;
- the absolute time range;
- active filters;
- essential fields or chart labels;
- enough context to understand what is being shown.

Before submission:

- crop unrelated browser content;
- hide passwords and private URLs;
- remove unrelated alerts and personal data;
- name the file using its evidence ID;
- record the screenshot in the evidence log;
- preserve the underlying event or query result when available.

A screenshot is not a substitute for searchable evidence.

---

10. Troubleshooting missing results

No alerts in the selected period

Check:

1. correct time range and time zone;
2. correct index/data view;
3. hidden filters;
4. Wazuh manager and indexer health;
5. collector status and cursor movement;
6. whether the event should have triggered a rule;
7. decoder and rule loading;
8. source record presence.

Fields are missing

Check:

- the expanded raw/full event;
- schema version;
- whether the JSON decoder recognised the event;
- recent field-name changes;
- index mappings;
- whether the source omitted the field.

Counts seem too high

Check:

- duplicate ingestion;
- grouped versus individual alerts;
- repeated rule matches for one event;
- the selected date histogram interval;
- whether several pods or sources are included.

Dashboard works but the VCC feed is empty

Check local collector health and enrolment state. Do not change pod identifiers or request parameters to test another pod. Escalate certificate, assignment or feed problems to the programme operator.

---

11. Guided exercise

Using Practice Lab 01:

1. Set an absolute UTC range covering the dataset.
2. Find the repeated authentication-failure alert.
3. Record its rule ID and source fields.
4. Filter by `svc-backup`.
5. Pivot to `203.0.113.44`.
6. Pivot to the successful session.
7. identify the authorization denial and account change;
8. identify the telemetry-health gap;
9. save a view;
10. complete the query journal and one evidence statement.

Required reflection

Explain why the successful login and later account change are stronger together than either event in isolation. Then explain how the telemetry gap limits conclusions about process activity.

---

12. Review questions

1. What is the difference between `wazuh-alerts-*` and `wazuh-archives-*`?
2. Why should an analyst use an absolute time range in a final report?
3. Name five useful VCC correlation fields.
4. Why might an event exist without an alert?
5. What is the difference between a WQL filter and a Discover search?
6. What should a screenshot contain to be useful evidence?
7. Why is an empty dashboard not proof that nothing happened?
8. Which step prevents an intern from accessing another pod’s telemetry?

Authoritative references

- Wazuh, *Navigating the Wazuh dashboard*.
- Wazuh, *Filtering data using Wazuh Query Language*.
- Wazuh, *Wazuh indexer indices*.
- Wazuh, *Log data analysis* and *Event logging*.
- Wazuh, *Wazuh agent connection*.
- `research/AUTHORITATIVE_SOURCE_REGISTER.md`.

Source: docs/setup/WORKSTATION_COMPATIBILITY.md

NeoLabs SOC L1 Workstation Compatibility

Supported baseline

The student Wazuh stack is designed for a personally controlled workstation used only for authorised NeoLabs/RIL training. Run the compatibility check before downloading images or starting the stack:

```
bash wazuh-stack/scripts/compatibility-check.sh
```

A supported workstation should provide:

Requirement	Minimum	Recommended
CPU architecture	64-bit x86_64 or arm64	x86_64
CPU capacity	4 logical cores	6 or more logical cores
System memory	8 GiB	12–16 GiB
Free disk space	25 GiB	50 GiB
Container runtime	Docker Engine/Desktop	Current supported Docker release
Compose	Docker Compose v2	Current supported v2 release
Linux kernel setting	`vm.max_map_count=262144`	`262144` or higher

The stack also needs Python 3, OpenSSL and curl for validation and secure enrolment.

Platform guidance

Ubuntu or another current Linux distribution

This is the preferred student platform. Docker must be running, and the learner's account must be permitted to use it. Set the indexer mapping limit before startup when the compatibility check reports a failure:

```
sudo sysctl -w vm.max_map_count=262144
```

Make the setting persistent using the operating system's approved sysctl configuration process.

Windows 11 with WSL2

Use an Ubuntu WSL2 distribution with Docker Desktop's WSL integration enabled. Store the repository inside the Linux filesystem, such as `~/neolabs-soc1-toolkit`, rather than under `/mnt/c/`; Linux filesystem storage provides more predictable permissions and container performance.

Do not run the stack directly from Git Bash, MSYS or Cygwin. Those shells do not provide the Linux permission and volume behaviour expected by the scripts.

macOS on Intel

Docker Desktop is supported when the machine meets the memory and disk requirements. Allocate enough Docker Desktop memory for the indexer, manager, dashboard and telemetry collector.

macOS on Apple Silicon

The scripts and learning materials work on arm64, but each pinned Wazuh container release must publish a compatible arm64 image. The compatibility check therefore reports Apple Silicon as a review warning rather than an unconditional pass. Use an x86_64 Linux or WSL2 workstation when the pinned release does not provide the required image architecture.

Unsupported configurations

The following configurations are outside the supported student baseline:

- 32-bit operating systems;
- Docker Toolbox or legacy Compose v1;
- production servers or shared company infrastructure;
- public cloud hosts exposed directly to the internet;
- systems with less than 8 GiB memory;
- systems where the learner does not control local Docker access;
- mobile devices and browser-only coding environments.

Pre-start checklist

1. Run `compatibility-check.sh` and resolve every failure.
2. Confirm that the repository is on a local, trusted filesystem.
3. Copy `.env.example` to `.env` and generate local secrets.
4. Run `preflight.sh` and `prepare-stack.sh`.
5. Start the stack and run `health-check.sh`.
6. Complete VCC enrolment only with a private, operator-issued token.

A warning may be accepted only when its risk is understood and documented. A failure must be corrected before the learner starts the Wazuh stack.

Source: docs/setup/BACKUP_AND_RECOVERY.md

NeoLabs SOC L1 Backup and Recovery

Purpose

The student Wazuh environment contains locally generated investigation data, dashboards, indexer data and manager state. A workstation failure should not require a learner to rebuild every investigation from the beginning.

The toolkit therefore provides a guarded Docker-volume backup and restore process. It is designed for local training data, not for production disaster recovery.

Security boundary

A toolkit backup deliberately excludes:

- `.env`;
- VCC bootstrap tokens;
- client private keys;
- client certificates and CA trust files;
- enrolment responses;
- private VCC endpoints;
- production data or credentials.

After restoring, the learner must complete a fresh operator-approved enrolment. This prevents copied backups from becoming reusable VCC access packages.

Create a backup

From the repository root:

```
bash wazuh-stack/scripts/backup.sh
```

The script performs the following actions:

1. confirms that Docker, Compose and the local `.env` file are available;
2. discovers the named volumes declared by the Wazuh Compose file;
3. stops the running stack to obtain a consistent point-in-time snapshot;
4. creates one compressed archive per existing volume;
5. calculates SHA-256 checksums;
6. writes a manifest and recovery notice;
7. restarts the stack unless `--no-restart` was supplied.

Use a specific destination when required:

```
bash wazuh-stack/scripts/backup.sh --output /secure/local/path/neolabs-backup
```

Keep the backup on encrypted, personally controlled storage. Do not upload it to a public repository or share it in Slack or WhatsApp.

Verify a backup

Always verify a backup before relying on it:

```
bash wazuh-stack/scripts/verify-backup.sh /path/to/backup
```

Verification checks the manifest version, checksum file, archive readability, presence of at least one volume and absence of common credential filenames.

Restore a backup

A restore replaces the contents of the matching Wazuh volumes. Stop the stack and confirm the backup path before continuing:

```
bash wazuh-stack/scripts/stop.sh
bash wazuh-stack/scripts/restore.sh /path/to/backup --force
```

The restore script:

- verifies checksums before modifying volumes;
- accepts only volume names declared in the current Compose file;
- refuses to run while stack services are active;
- recreates and repopulates the declared volumes;
- removes stale local VCC collector scope and cursor state;
- does not restore enrolment credentials.

After restoration:

```
bash wazuh-stack/scripts/compatibility-check.sh
bash wazuh-stack/scripts/start.sh
bash wazuh-stack/scripts/health-check.sh
```

Then request a new enrolment token from the programme operator and complete the normal enrolment workflow.

Rehearsal

The repository CI performs a synthetic Docker-volume recovery rehearsal using:

```
bash wazuh-stack/tests/backup-restore-rehearsal.sh
```

The rehearsal creates a temporary volume, writes a synthetic marker, archives it, verifies its checksum, deletes and recreates the volume, restores the archive and confirms that the marker survived. It never reads the learner's Wazuh data.

Recovery limitations

A successful archive verification does not prove that every future Wazuh release can read data created by an older release. Restore into the same pinned toolkit version first. Complete a documented Wazuh upgrade only after the restored stack is healthy.

Backups are not a substitute for GitHub submissions. Investigation reports, evidence logs and query journals should still be committed to the approved assignment repository after removing credentials, private URLs and sensitive evidence.

Source: troubleshooting/WAZUH_SETUP_AND_TROUBLESHOOTING_GUIDE.md

NeoLabs Wazuh Setup and Troubleshooting Guide

Tested baseline: Wazuh 4.14.7
Guide version: 0.2-draft
Review date: 1 August 2026
Scope: Learner-owned workstation and authorised VCC synthetic telemetry only

> Troubleshoot from the source toward the dashboard. Changing several settings at once can hide the real cause and create a second problem.

1. Golden troubleshooting sequence

```
Host requirements
  → Docker daemon
  → local files and permissions
  → generated Wazuh configuration
  → container startup
  → component health
  → VCC enrolment and mTLS
  → collector output
  → manager file monitoring
  → decoding and rules
  → Filebeat/indexer
  → dashboard data view, time and filters
```

Record every command and result in the query journal. Remove credentials and private URLs before submitting evidence.

---

2. Safe commands first

Run from `wazuh-stack/`.

Preflight

```
bash scripts/preflight.sh
```

Service status

```
bash scripts/health-check.sh
```

Compose process list

```
docker compose --env-file .env ps
```

Recent service logs

```
docker compose --env-file .env logs --since=10m \
  wazuh.manager wazuh.indexer wazuh.dashboard vcc.telemetry.collector
```

Before sharing logs, review them for:

- private VCC endpoints;
- usernames or personal information;
- certificate details;
- environment values;
- raw application data outside the assignment.

Do not use `docker inspect` to publish complete environment blocks.

Part I — Host and Docker problems

3. `docker: command not found`

Meaning

Docker Engine or Docker Desktop is not installed, or the command is not on the current PATH.

Checks

```
docker --version
docker compose version
```

Resolution

Install Docker from the official Docker instructions for the learner’s operating system. Use the Compose plugin, not an unverified third-party package.

Escalate when

- the learner does not have permission to install system software;
- the device is managed by an organisation;
- hardware virtualisation is disabled and requires administrator access.

4. Docker daemon not running

Typical message:

```
Cannot connect to the Docker daemon
```

Checks

```
docker info
```

On Docker Desktop, confirm the application is running. On Linux, an authorised administrator may check the Docker service.

Caution

Do not solve this by exposing an unauthenticated Docker TCP socket. Docker daemon access provides extensive control over the local host.

5. Permission denied accessing Docker

Typical Linux message:

```
permission denied while trying to connect to the Docker daemon socket
```

The user lacks access to the local Docker daemon.

Follow the approved system-administration procedure. Membership in the Docker group is effectively privileged on that workstation and should not be treated as a harmless application permission.

6. Insufficient memory or storage

Possible symptoms:

- indexer repeatedly restarts;
- dashboard remains unavailable;
- containers are killed with exit code 137;
- searches are extremely slow;
- disk watermark or read-only index errors appear.

Checks

```
docker stats --no-stream
docker system df
df -h
```

The all-in-one training baseline should have approximately 4 CPU cores, 8 GiB RAM and 50 GB available storage.

Resolution

- stop unrelated heavy applications;
- allocate enough resources to Docker Desktop;
- free approved local storage;
- do not delete Wazuh volumes unless the reset procedure is authorised;
- report repeated disk-watermark errors to the mentor.

7. `vm.max_map_count` too low

The indexer may fail with a message related to virtual memory areas.

Check

```
cat /proc/sys/vm/max_map_count
```

Required baseline:

```
262144 or higher
```

Use the operating-system-specific official OpenSearch/Wazuh guidance. A learner should not change shared or managed systems without permission.

Part II — Local files and preparation

8. Missing `.env`

Typical preflight error:

```
Missing wazuh-stack/.env
```

Run:

```
bash scripts/generate-local-secrets.sh
```

The script refuses to overwrite an existing `.env` because that could destroy valid local credentials or configuration.

9. Placeholder password remains

Preflight rejects values beginning with:

```
CHANGE_ME
```

Use the secret-generation script. Do not replace placeholders with a short shared classroom password.

10. `.env` permissions are too broad

Expected mode:

```
600
```

Check:

```
stat -c '%a %n' .env
```

Fix on a compatible Unix-like system:

```
chmod 600 .env
```

On platforms that do not expose Unix permissions in the same way, follow the tested platform-specific cohort guidance.

11. `prepare-stack.sh` cannot verify the Wazuh commit

The script checks that the official tag resolves to the exact reviewed commit.

Do not bypass the comparison. Possible causes include:

- the `.env` pin was edited;
- an incomplete or incorrect tag was configured;
- the local upstream checkout is damaged;
- the reviewed baseline has changed.

Safe recovery

Remove only the ignored upstream state directory after confirming no local evidence is stored there, then rerun preparation. Do not change the expected commit to make the error disappear without technical review.

12. Certificate generation fails

Possible causes:

- Docker cannot pull the official generator image;
- the generated directory has incorrect ownership;
- stale generated files conflict;
- insufficient disk space;
- network access to the container registry is unavailable.

Checks

```
docker pull wazuh/wazuh-certs-generator:0.0.2
ls -la generated
```

Use the exact image referenced by the reviewed official Wazuh release. Do not substitute an unknown certificate generator.

Part III — Container startup

13. Compose configuration error

Validate without starting:

```
docker compose --env-file .env config --quiet
```

Common causes:

- missing environment variables;
- malformed YAML;
- missing generated bind-mount files;
- a host path created as a directory when a file was expected;
- unsupported Compose version.

14. Dashboard port already in use

Typical error:

```
address already in use
```

Check which approved local process uses the port. Change `WAZUH_DASHBOARD_PORT` to another unused high port only if the cohort guidance permits it.

Keep:

```
WAZUH_DASHBOARD_BIND=127.0.0.1
```

Do not change the bind address to `0.0.0.0` merely to fix a local browser problem.

15. Manager unhealthy

Check:

```
docker compose --env-file .env logs --since=10m wazuh.manager
```

Possible causes:

- invalid `ossec.conf` XML;
- unreadable mounted certificate;
- indexer connection failure;
- invalid local rules;
- volume permission issue;
- resource exhaustion.

Rule/config validation

Inspect manager startup messages for the exact file and line. Do not delete the rule file blindly. Restore the reviewed file or test the specific change.

16. Indexer unhealthy

Check:

```
docker compose --env-file .env logs --since=10m wazuh.indexer
```

Common causes:

- `vm.max_map_count` too low;
- memory exhaustion;
- certificate mismatch;
- invalid internal-user password hashes;
- corrupted or incompatible data volume;
- disk watermark.

Do not delete `wazuh-indexer-data` as a first response. It contains the local searchable investigation data.

17. Dashboard unhealthy

Check:

```
docker compose --env-file .env logs --since=10m wazuh.dashboard
```

Common causes:

- indexer not healthy;
- wrong indexer password;
- wrong Wazuh API password;
- missing certificate mount;
- dashboard waiting for dependencies;
- insufficient memory.

Follow dependency order: indexer and manager must be healthy before expecting the dashboard to become ready.

Part IV — Browser and login

18. Browser cannot open the dashboard

Confirm:

```
bash scripts/health-check.sh
```

Then use exactly the address printed by the script.

Checks:

- correct port;
- `https`, not `http`;
- loopback address;
- no corporate proxy rewriting local traffic;
- dashboard container healthy.

19. Certificate warning

The local Wazuh dashboard uses generated training certificates. Verify:

- the address is the local loopback address;
- the certificate belongs to the local generated stack;
- the programme-approved browser procedure allows proceeding.

Never generalise this behaviour to public or unknown websites.

20. Dashboard password rejected

Do not paste the password into screenshots or chat.

Possible causes:

- `.env` was regenerated after the indexer security configuration was prepared;
- preparation was run with one password and startup with another;
- browser password manager inserted an old value;
- local index data belongs to a previous configuration.

The local `.env`, generated `internal_users.yml` and running containers must belong to the same preparation cycle. Use the documented reset/rebuild procedure only after preserving required evidence.

Part V — VCC enrolment

21. Enrolment base URL is empty

Set the operator-provided HTTPS base URL in `.env`:

```
VCC_ENROLMENT_BASE_URL=https://<approved-host>:8443
```

Do not add a pod path or query parameter.

22. Enrolment CA certificate missing

Expected path:

```
secrets/vcc/enrolment-ca.crt
```

The CA must come from the NeoLabs operator through the approved channel. Do not download or replace it from an unverified message or website.

23. TLS verification failed during enrolment

Possible causes:

- incorrect CA certificate;
- hostname does not match the server certificate;
- connecting to the wrong port or host;
- system clock incorrect;
- intercepting proxy;
- expired server certificate.

Do not disable TLS verification. Escalate the hostname and certificate error to the operator.

24. Bootstrap token rejected

The client intentionally returns a general message for invalid, expired, consumed or mismatched tokens.

Possible causes:

- token expired;
- token was already used;
- a replacement token invalidated it;
- assignment was revoked;
- whitespace or line wrapping altered the token;
- the token belongs to another installation workflow.

Request a new token through the approved channel. Do not ask another intern for theirs.

25. Existing local enrolment detected

The client refuses to overwrite active material.

Correct procedure:

1. contact the operator;
2. identify the existing credential ID from local enrolment metadata without publishing it;
3. obtain server-side revocation;
4. confirm whether the assignment changes;
5. rerun enrolment with explicit replacement only after approval.

Deleting the certificate locally does not revoke it on the server.

26. Enrolment succeeds but collector shows authentication error

Possible causes:

- credential revoked after issuance;
- certificate expired;
- telemetry URL changed;
- installation ID file changed;
- mTLS Nginx route not configured correctly;
- client and server clocks differ significantly.

Do not regenerate the installation ID. Preserve the local health record and ask the operator to check the credential and audit events.

Part VI — Collector problems

27. Collector status `unenrolled`

This is expected before valid endpoint and certificate files exist.

Check:

```
ls -l secrets/vcc
cat state/collector-health.json
```

Do not print private key contents.

28. Collector status `connection_error`

Possible causes:

- endpoint unavailable;
- DNS failure;
- firewall or proxy block;
- server certificate verification failure;
- timeout.

Use only non-secret connectivity checks approved by the operator. Do not use insecure curl flags against the VCC endpoint in submitted work.

29. Collector status `authentication_error`

The API returned HTTP 401 or 403.

Likely causes:

- missing or invalid client certificate;
- revocation;
- installation ID mismatch;
- assignment revoked;
- certificate expired.

This requires operator-side audit review. Editing `POD_LABEL` cannot fix authentication.

30. Collector status `validation_error`

The collector rejected the response.

Possible causes:

- missing required field;
- `synthetic` is not exactly `true`;
- event `pod_id` differs from the server-issued pod;
- response omitted `X-VCC-Pod-ID`;
- invalid NDJSON;
- oversized response;
- server-issued pod changed without credential reset.

Treat a wrong-pod event as a security stop condition. Preserve the private evidence and notify the programme operator immediately.

31. Cursor not moving

A cursor can remain unchanged when no new events exist. Check:

- collector health says the poll succeeded;
- event count returned was zero;
- assignment scenario is active;
- cursor file modification time;
- operator exporter health.

Do not delete the cursor to force replay unless the operator approves. Replaying a large range can duplicate local analysis work.

Part VII — Manager ingestion and detection

32. Collector writes events but Wazuh shows none

Validate the shared data path inside containers without exposing event content unnecessarily.

Questions:

1. Is the collector output file present?
2. Does the manager mount the same named volume?
3. Does the manager configuration monitor the exact path?
4. Is `log_format=json` set?
5. Did the manager reload the configuration?
6. Are events valid one-object-per-line NDJSON?

33. JSON decoder does not expose fields

Use `wazuh-logtest` in the local manager with one approved synthetic record.

Check:

- valid JSON;
- no extra prefix before `{`;
- field types are expected;
- schema version is supported;
- nested fields match the rule paths;
- the event fits within size limits.

A JSON record being syntactically valid does not guarantee its schema matches the rules.

34. Rule does not match

Check:

- parent rule matched;
- field name and case;
- regex anchors;
- expected string versus Boolean representation;
- custom rule file loaded;
- rule ID does not collide;
- level and grouping;
- correlation timeframe and frequency;
- correct static/dynamic correlation field.

Test one simple event first, then the sequence.

35. Repeated-event rule does not trigger

Correlation rules require events to arrive within the configured timeframe and satisfy grouping conditions.

Check:

- event ingestion order;
- Wazuh analysis timestamp behaviour;
- same-user or same-field values are populated consistently;
- frequency count includes the intended parent rule;
- the test session is not affected by prior cached events;
- malformed or rejected events did not break the sequence.

36. Alert exists in manager but not dashboard

Check:

1. manager alert file;
2. Filebeat logs;
3. certificate and indexer connectivity;
4. indexer health;
5. index existence;
6. dashboard data view;
7. absolute time range;
8. hidden filters.

This indicates the detection stage worked and the problem is later in the pipeline.

Part VIII — Indexer and dashboard investigation problems

37. Indexer cluster red or read-only

Possible causes:

- disk pressure;
- unavailable shard;
- corrupted volume;
- memory instability;
- incompatible restored data.

Do not run destructive index commands as a learner. Preserve logs and escalate.

38. Query returns zero results unexpectedly

Check:

- correct index pattern;
- absolute UTC range;
- event versus ingest timestamp;
- exact field path;
- keyword versus analysed mapping;
- active dashboard filters;
- pod and synthetic values;
- alert level and archive availability.

Record the zero-result query and the source-health checks before calling it evidence.

39. Fields appear under a different path

Wazuh and index mappings may nest source fields under `data` or another source-specific object.

Open a real event, inspect the field list and use the actual indexed path. Update the query journal. Do not silently change the training rule without confirming the decoder and mapping.

40. Times appear shifted

Check:

- browser time-zone setting;
- dashboard advanced settings;
- source `event_time`;
- indexed `timestamp`;
- collector `ingest_time`;
- host clock.

Build the report timeline in UTC and document conversions.

Part IX — Reset and recovery

41. Stop without deleting data

```
bash scripts/stop.sh
```

42. Destructive reset

First run without confirmation to display the warning:

```
bash scripts/reset.sh
```

Then, only after preserving required work:

```
bash scripts/reset.sh --confirm-destroy-local-data
```

This removes Wazuh volumes, generated configuration and local telemetry state. It retains enrolment material unless the additional approved flag is used.

43. Removing enrolment material

Server-side revocation must happen first. Then:

```
bash scripts/reset.sh \
  --confirm-destroy-local-data \
  --include-enrolment
```

Confirm revocation with the operator. Local deletion alone is not a security control.

Part X — Escalation package

When troubleshooting cannot be completed safely, provide:

```
Workstation OS and version:
Docker and Compose versions:
Wazuh version:
Exact failing step:
First error time in UTC:
Service health summary:
Relevant redacted error lines:
Commands already run:
Whether the system ever worked:
Recent approved changes:
Assigned pod as shown by local state:
Credential ID reference, not private certificate/key:
Impact on investigation:
```

Do not include:

- `.env` contents;
- tokens;
- private keys;
- complete certificate dumps;
- private VCC URLs in public issues;
- unrelated student or pod evidence.

Quick decision tree

```
Dashboard unavailable?
  ├─ Compose invalid → fix local configuration
  ├─ Indexer unhealthy → host/kernel/memory/cert/storage checks
  ├─ Manager unhealthy → XML/rule/cert/indexer checks
  └─ Dashboard only unhealthy → API/indexer/password/cert checks

Dashboard available but no VCC events?
  ├─ Collector unenrolled → complete approved enrolment
  ├─ Authentication error → operator credential/audit review
  ├─ Validation error → security stop; check wrong-pod/schema response
  ├─ No collector output → exporter/feed/connectivity review
  ├─ Output exists, no decoded event → manager localfile/JSON review
  ├─ Decoded event, no alert → rule/level/correlation review
  ├─ Manager alert exists, no index → Filebeat/indexer review
  └─ Indexed alert exists, not visible → data view/time/filter review
```

Authoritative references

- Wazuh official documentation: Docker deployment, component requirements, server logs, JSON decoder, custom rules, `wazuh-logtest`, indexer and dashboard troubleshooting.
- OpenSearch documentation: bootstrap checks, disk watermarks, cluster health and mappings.
- Docker documentation: daemon, Compose, resource controls, volumes and security.
- `wazuh-stack/README.md`.
- `research/AUTHORITATIVE_SOURCE_REGISTER.md`.

Source: wazuh-stack/README.md

NeoLabs Student Wazuh Stack

Purpose

This directory provides a repeatable local Wazuh manager, indexer and dashboard for authorised NeoLabs SOC Level 1 training. A separate hardened collector can receive only the synthetic telemetry issued to the learner's assigned VCC pod.

The baseline follows the official Wazuh single-node container topology and pins:

- Wazuh release `4.14.7`;
- official Docker tag `v4.14.7`;
- verified upstream commit `adcc5b57d2f7edfcbe6c399272dc76fbdf12b623`.

Changing the version requires release-note review, regenerated configuration, rule tests, Compose validation and a new recorded review date.

Current status

The stack is a reviewed **Version 1 release candidate** on the toolkit feature branch. The local manager, indexer, dashboard, collector, enrolment client, health checks, reset controls, backup/restore workflow, compatibility assessment, starter rules and CI tests are implemented.

The corresponding isolated VCC control-plane rehearsal passed:

- certificate-signing request exchange;
- single-use bootstrap-token enforcement;
- two separate intern-to-pod assignments;
- server-enforced pod isolation;
- rejection of client-selected pod scope;
- credential revocation;
- assignment revocation.

The repository also passes collector unit tests, Compose validation, secret-boundary checks and a synthetic Docker-volume backup/restore rehearsal. Automated workstation validation runs on Linux CI. WSL2 and macOS deployment profiles are documented in `../docs/setup/WORKSTATION_COMPATIBILITY.md`, but should still be checked on the actual cohort machines during controlled rollout.

This branch has not been merged or deployed. Students should use it only after NeoLabs operator approval and receipt of their individual enrolment details.

Required workstation capacity

Run the automated assessment first:

```
bash scripts/compatibility-check.sh
```

For the all-in-one Wazuh topology, plan for approximately:

- 4 CPU cores;
- 8 GiB RAM minimum, with 12–16 GiB recommended;
- 25 GiB free storage minimum, with 50 GiB recommended;
- Docker Engine or Docker Desktop with Compose v2;
- Linux, WSL2 or a reviewed Docker Desktop environment;
- `vm.max_map_count` of at least `262144` where the host exposes that setting;
- Python 3, OpenSSL and curl.

Smaller systems may start but can become unstable or too slow for investigations.

Security design

Network exposure

- The Wazuh indexer is not published to the host.
- The Wazuh server API is not published to the host.
- The dashboard binds to `127.0.0.1:8443` by default.
- Wazuh components use an internal-only Compose network.
- Only the VCC telemetry collector receives an outbound network path.

Pod isolation

The learner does not choose an authorised pod by editing `POD_LABEL`, a URL or a request parameter.

1. A programme operator records the intern-to-pod assignment in the VCC control plane.
2. The operator issues a short-lived, single-use bootstrap token.
3. The enrolment client creates a local private key and sends only a certificate signing request.
4. The control plane consumes the token and issues a client certificate bound to the recorded assignment.
5. Nginx verifies the client certificate for every telemetry request.
6. The control plane derives pod scope from the active certificate and assignment.
7. The collector rejects any event whose `pod_id` differs from the server-issued pod.
8. Reassignment requires server-side revocation and explicit local re-enrolment.

A shared pod password is intentionally not used.

Files that must remain private

Never commit or send through a public channel:

- `.env`;
- bootstrap token files;
- `secrets/vcc/client.key`;
- issued client certificates when programme policy treats them as confidential;
- VCC private URLs;
- `state/enrolment.json`;
- raw evidence containing personal information or another pod's data;
- generated backups.

The operator-issued enrolment CA certificate is public-key material, but it must still be delivered through an approved channel so the learner can verify that it belongs to the real VCC lab.

Setup workflow

Run commands from `wazuh-stack/`.

1. Check the workstation

```
bash scripts/compatibility-check.sh
```

Resolve every failure before continuing. Platform-specific guidance is available in `../docs/setup/WORKSTATION_COMPATIBILITY.md`.

2. Generate local secrets

```
bash scripts/generate-local-secrets.sh
```

This creates `.env`, an installation identifier and protected local directories. Password values are not printed.

3. Configure the enrolment endpoint

The operator supplies:

- the HTTPS enrolment base URL;
- the VCC enrolment CA certificate;
- a short-lived bootstrap token file when the intern is ready to enrol.

Place the CA at:

```
secrets/vcc/enrolment-ca.crt
```

Set the operator-provided URL in `.env`:

```
VCC_ENROLMENT_BASE_URL=https://soc.lab.example.invalid:8443
```

Do not add a pod ID to this URL.

4. Prepare the pinned Wazuh configuration

```
bash scripts/prepare-stack.sh
```

The script:

- clones the official `wazuh/wazuh-docker` repository into ignored local state;
- checks out the exact pinned tag;
- verifies the full expected commit SHA;
- copies the official single-node configuration;
- generates local indexer password hashes;
- generates the Wazuh TLS certificate set;
- adds the NeoLabs VCC NDJSON input configuration.

No Wazuh service is started by this step.

5. Run preflight validation

```
bash scripts/preflight.sh
```

Preflight checks Docker, Compose, local file permissions, placeholder passwords, pinned versions, dashboard binding, memory guidance and `vm.max_map_count` where available.

6. Enrol to the assigned VCC pod

Protect the operator-issued token file:

```
chmod 600 /path/to/bootstrap-token.txt
```

Then run:

```
bash scripts/enrol-vcc.sh --token-file /path/to/bootstrap-token.txt
```

The client does not send a learner-selected pod. It stores the server-issued pod and credential metadata locally without printing private key or certificate contents.

7. Start and validate the stack

```
bash scripts/start.sh
```

The startup script validates the generated files, checks the Compose model, builds the collector, starts services and waits for health checks.

View current health later with:

```
bash scripts/health-check.sh
```

The collector is considered healthy while waiting for enrolment, but Wazuh will not receive VCC telemetry until valid credentials and an endpoint exist.

8. Back up local Wazuh data

```
bash scripts/backup.sh
```

The stack is stopped briefly for a consistent archive and restarted afterward. Backups exclude `.env`, VCC tokens, private keys, certificates and private endpoints. Verify a backup with:

```
bash scripts/verify-backup.sh /path/to/backup
```

The complete recovery procedure is documented in `../docs/setup/BACKUP_AND_RECOVERY.md`.

9. Stop without deleting data

```
bash scripts/stop.sh
```

This retains Wazuh volumes, local telemetry and enrolment credentials.

10. Restore local Wazuh data

Stop the stack, verify the selected backup and restore only after confirming the destructive replacement:

```
bash scripts/restore.sh /path/to/backup --force
```

Credential material is intentionally not restored. Complete a new operator-approved enrolment after recovery.

11. Destructive local reset

Review the warning first:

```
bash scripts/reset.sh
```

To delete local Wazuh data and generated configuration:

```
bash scripts/reset.sh --confirm-destroy-local-data
```

Removing enrolment material additionally requires confirmed server-side revocation:

```
bash scripts/reset.sh --confirm-destroy-local-data --include-enrolment
```

Deleting local certificate files alone does not revoke the server credential.

VCC telemetry format

The collector accepts newline-delimited JSON and requires at least:

```
{
  "schema_version": "1.0",
  "event_id": "evt-example-001",
  "event_time": "2026-08-01T09:14:21Z",
  "pod_id": "pod-03",
  "event_type": "authentication",
  "synthetic": true
}
```

The collector fails closed when:

- the event is not marked synthetic;
- required fields are missing;
- an event carries a different pod from the server-issued response header;
- the server-issued pod changes without credential reset;
- TLS verification fails;
- the response exceeds the configured size or event-count limit.

Initial NeoLabs rules

`config/rules/neolabs_vcc_rules.xml` contains training detections for:

- authentication failure;
- repeated failures by account;
- repeated failures by source;
- successful authentication;

- success following earlier failures;
- sensitive account changes;
- protected access-control denials;
- telemetry quality and availability problems.

These are starter rules, not proof of compromise. Analysts must inspect the underlying records, context and visibility limitations.

Troubleshooting order

When events are missing, check in this order:

1. local Compose and container health;
2. enrolment state and certificate expiry;
3. collector health JSON and cursor movement;
4. HTTPS/mTLS connectivity without printing secrets;
5. presence of records in the shared `vcc_telemetry` volume;
6. Wazuh manager file monitoring;
7. JSON decoding and required fields;
8. rule loading and `wazuh-logtest` validation;
9. indexer health and dashboard time filters.

Use `../troubleshooting/WAZUH_SETUP_AND_TROUBLESHOOTING_GUIDE.md` for the full decision tree.

References

- Official Wazuh documentation and `wazuh/wazuh-docker` release repository.
- Docker Compose and Docker Engine security documentation.
- `../research/AUTHORITATIVE_SOURCE_REGISTER.md`.
- VCC control-plane architecture and implementation in the private VCC Security Lab repository.